

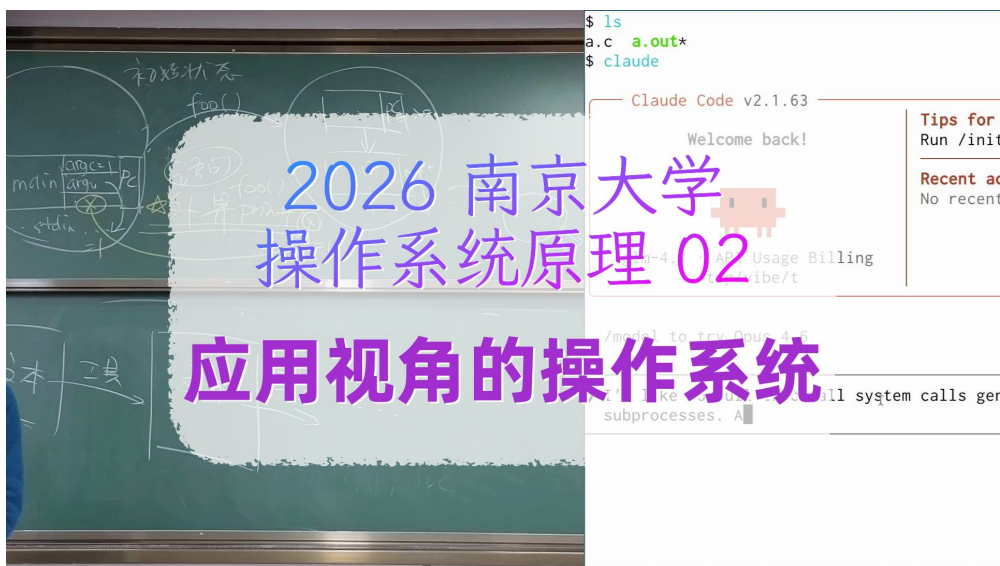
课程笔记

应用视角的操作系统

南京大学 操作系统原理 2026 春季学期 第 02 讲

lecture-to-notes

2026 年 3 月 24 日



视频作者/频道：绿导师原谅你了（蒋炎岩 jyy）

发布日期：2026 年春季学期

视频时长：1 小时 28 分钟

视频链接：<https://www.bilibili.com/video/BV1KNPHzPEra>

目录

1 什么是程序？	2
1.1 为什么选 C 语言？	2
1.2 C ~ SimpleC：每行只做一小件事	3
1.3 状态机执行追踪	4
1.4 理解函数调用：LLM 的极限测试	4
1.5 程序状态机的形式化定义	5
1.6 从递归到迭代：汉诺塔的状态机视角	6
1.7 本章小结	7
2 编译与优化	7
2.1 编译器 = 状态机翻译器	7
2.2 编译器优化	8
2.3 本章小结	9
3 最小 Hello World	9
3.1 标准 hello.c 的真面目	9
3.2 minimal.S：不依赖任何库的最小程序	10
3.3 系统调用：请求操作系统服务	11
3.4 本章小结	12
4 开盒真实应用程序	12
4.1 操作系统面前众生平等	12
4.2 strace：系统调用追踪	13
4.3 开盒 gcc：编译器到底做了什么？	13
4.4 本章小结	14
5 总结与延伸	14
5.1 讲者的核心信息	14
5.2 结构化提炼	15
5.3 跨章节关联	15
5.4 与第 01 讲的关联	15
5.5 开放问题	15
5.6 拓展阅读	16

1 什么是程序？

jyy 开宗明义：“从今天起我们的主角就是**程序**。操作系统就是一组 API，从应用程序的视角来理解操作系统，就是理解程序如何使用这些 API。”

回顾操作系统的定义（OSTEP）：

Operating System: A body of software, in fact, that is responsible for making it easy to run programs (even allowing you to seemingly run many at the same time), allowing programs to share memory, enabling programs to interact with devices, and other fun stuff like that.

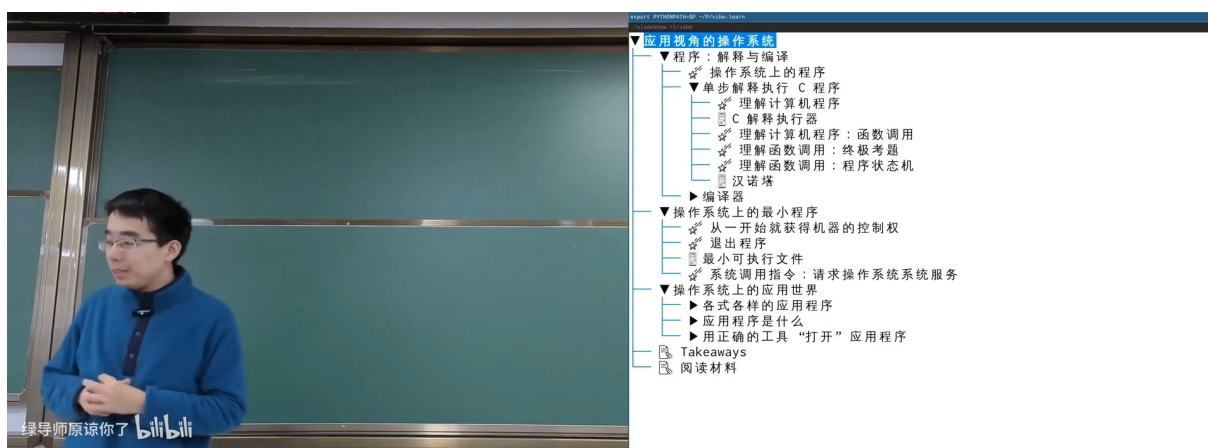


图 1: 本讲大纲：从理解程序到开箱真实应用¹

《操作系统》课会讲清楚什么是程序

精确到每一个 bit 都是 well-defined 的。精确也是为“形式定义”的第一步。操作系统是硬件和软件中间的那个中间层，我们选一个离操作系统尽可能近的编程语言——C 语言——来作为理解程序的工具。

1.1 为什么选 C 语言？

jyy 解释了选择 C 语言的理由链条：

- 汇编可以直接和操作系统交互，但不能用它开发应用程序
- 自然语言（甚至 AI prompt）是最高级的“语言”，但太模糊
- C 语言介于两者之间：“离机器最近的，但又可以写应用程序的语言”
- 这也是最早期整个 UNIX 应用生态被构建起来时使用的语言

¹ 视频画面时间区间：00:00:15–00:00:45。

1.2 C ~ SimpleC: 每行只做一小件事

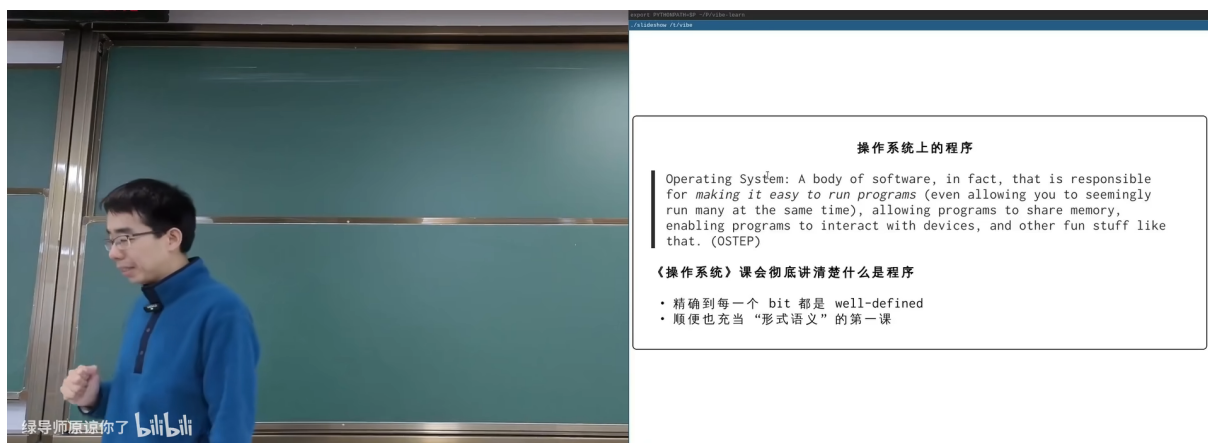


图 2: 操作系统上的程序: OSTEP 定义与 SimpleC 理念²

jyy 的核心观察: 虽然 C 提供了很多方便写法 (长表达式、嵌套函数调用), “但是你总是可以把任何一个任意复杂的表达式改写成每一行都只做一小件事情的形式”。

这就是 SimpleC——一个 C Intermediate Language transpiler:

```
1 while (1) {  
2     stmt = fetch_statement();  
3     decode_and_execute(stmt);  
4 }
```

Listing 1: SimpleC 解释器核心循环: 取指令-解码-执行

程序的本质

程序 = 一个不断“取指令、解码、执行”的循环。这和 CPU 做的事情完全一样——区别只是抽象层的高低。SimpleC 让你能亲手看到程序执行的每一步状态变化。

jyy 还提到了一个有趣的实验: “我们让 LLM 改写了 `printf`, 调试一下试试吧!”——用 AI 重新审视经典系统代码。

相关工具

- PicoC: <https://github.com/jpoirier/picoc> — a very small C interpreter for scripting
- CIL: <https://cil-project.github.io/cil/> — C Intermediate Language, 学术级的 C 分析框架

²视频画面时间区间: 00:01:45–00:02:30。

1.3 状态机执行追踪

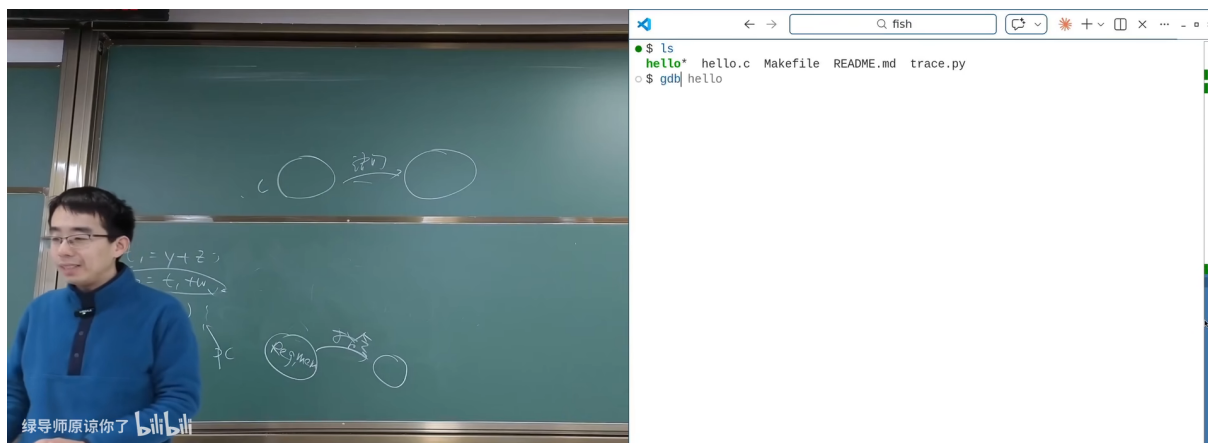


图 3: SimpleC 的状态追踪：每一步 Transition 和 State 都可观测³

jyy 用 SimpleC 演示了 `hello.c` 的逐步执行。每一步输出：

- **Transition**: 当前执行的源码位置和语句（如 `hello.c:89 -> d = va_arg(ap, int)`）
- **State**: 所有活跃变量的值（如 `d = 42, r = 12767, digit = 32767`）

从“执行指令”到“状态转移”的思维转换

很多人理解程序执行为“一行行运行代码”。但状态机视角要求关注的不是“执行了什么”，而是“状态如何变化”。这是理解操作系统的关键——操作系统管理的就是多个状态机的并发执行和资源共享。

1.4 理解函数调用：LLM 的极限测试

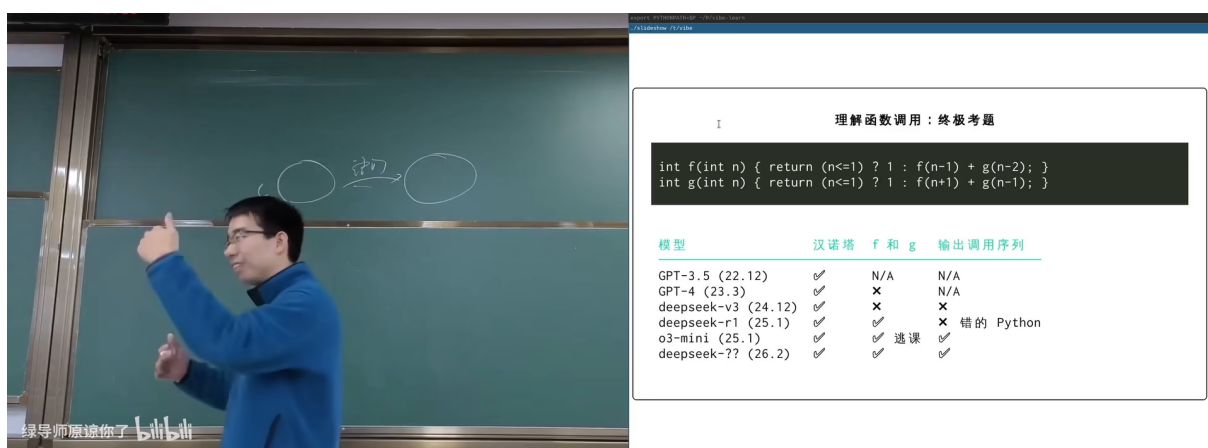


图 4: 各模型在函数调用推理上的表现对比⁴

jyy 用一个嵌套递归函数测试了多个 LLM 的推理能力：

³视频画面时间区间：00:11:00–00:11:45。

⁴视频画面时间区间：00:16:30–00:17:15。

```

1 int f(int n) { return (n<1) ? f(n-1) + g(n-2) ; }
2 int g(int n) { return (n<1) ? f(n-1) + g(n-1) ; }

```

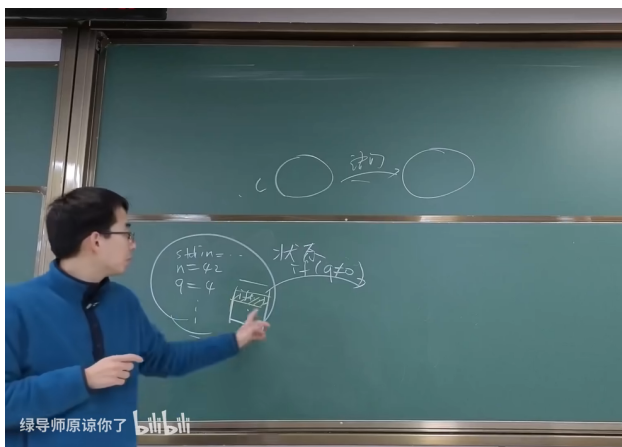
Listing 2: 测试题：相互递归

模型	逻辑正确	输出调用序列	备注
GPT-3.5 (22.12)	N/A	N/A	
GPT-4 (23.3)	×	N/A	
deepseek-v3 (24.12)	✓	✓	
deepseek-r1 (25.1)	✓	✓	输出 Python 辅助
o3-mini (25.1)	✓	✓	
deepseek-r7 (26.2)	✓	✓	

AI 推理能力的进化速度

从 2022 年底 GPT-3.5 完全无法处理，到 2024 年底 deepseek-v3 已经能正确推理——仅两年。但注意：这些模型需要“输出 Python 辅助计算”才做到，并非纯逻辑推理。这说明“理解程序执行”仍是 AI 的挑战之一。

1.5 程序状态机的形式化定义



理解函数调用：程序状态机

状态机是拥有**严格数学定义**的对象。这意味着你可以把定义写出来，并且用**数学严格**的方法理解它 -- 形式化方法

状态

- `[StackFrame, StackFrame, ...]` + 全局变量
- 状态 ≠ “变量 + 栈 + PC”

初始状态

- 仅有一个 `StackFrame(main, argv, argv, PC=0)`
- 全局变量全部为初始值

状态迁移

- 执行 `frames[-1].PC` 处的语句 (假设是 SimpleC)
- `f(x): stack[-1].next_pc(); stack.push(x=x, PC=f)`
 - 这一步是**完全机械**的操作
 - 许多体系结构都提供完成这个操作的 `call` 指令

图 5: 程序状态机：状态、初始状态、状态迁移的形式化定义⁵

jyy 强调：“当我们说所有的一切都是有严格数学定义的时候，我们的状态机是有一个初始状态的。”

⁵ 视频画面时间区间：00:22:00–00:22:45。

程序状态机三要素

状态 S : {StackFrame, StackFrame, ...} + 全局变量 + 代码 + “堆” + PC

初始状态 S_0 : 仅有一个 StackFrame = (main, argc, argv, PC=0), 加上所有全局变量的初始值

状态迁移 $S \rightarrow S'$: 执行 frames[-1].PC 处的语句

- 普通语句: push/pushv/PC++
- 函数调用 call: 压入新 StackFrame
- 函数返回 return: 弹出当前 StackFrame

1.6 从递归到迭代: 汉诺塔的状态机视角

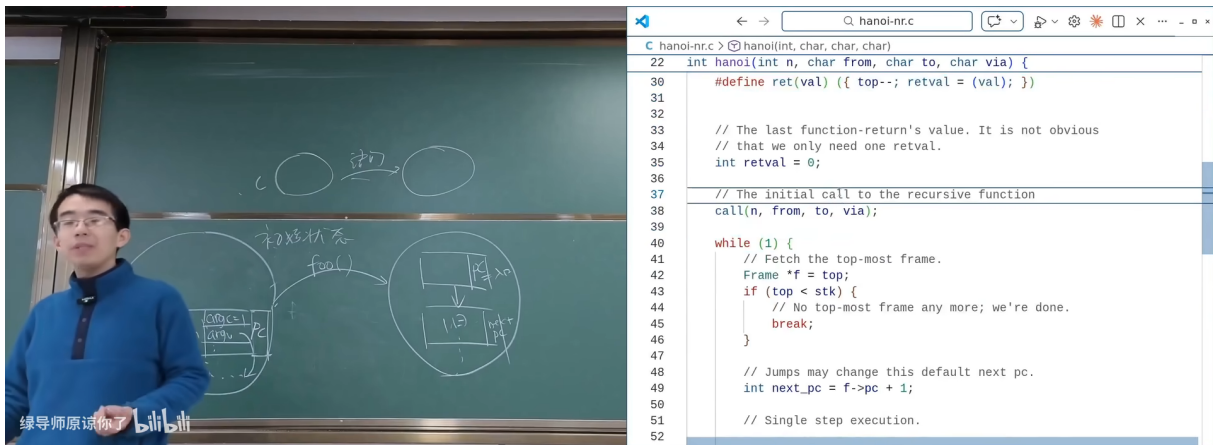


图 6: hanoi-nr.c: 用显式栈模拟递归调用⁶

jyy 展示了将递归汉诺塔转写为迭代的核心技巧——**显式管理调用栈**:

```
1 #define ret(val) ({ top--; retval = (val); })
2
3 while (1) {
4     Frame *f = top;
5     if (top == stk) break; // Stack empty -> done
6     int next_pc = f->pc++;
7     // Single step execution...
8 }
```

Listing 3: 递归转迭代: 显式栈帧管理

递归转迭代的通用方法

任何递归程序都可以通过显式管理调用栈转为迭代——这不是编程技巧, 而是状态机理论的直接推论。递归只是“栈帧的隐式管理”。操作系统本身就在显式管理所有进程的“栈”——进程调度的本质就是在多个状态机之间切换。

⁶视频画面时间区间: 00:26:00-00:26:45。

1.7 本章小结

程序的本质是状态机。C 语言可以被简化为 SimpleC（每行一条简单操作），SimpleC 的执行就是状态机的迁移。函数调用是栈帧的 push/pop，递归转迭代是栈的显式化。这个框架为理解编译器和操作系统奠定了基础。

2 编译与优化

jyy 原话：“既然我们讲清楚了什么是程序，那我们就可以说什么是编译器了。”

2.1 编译器 = 状态机翻译器

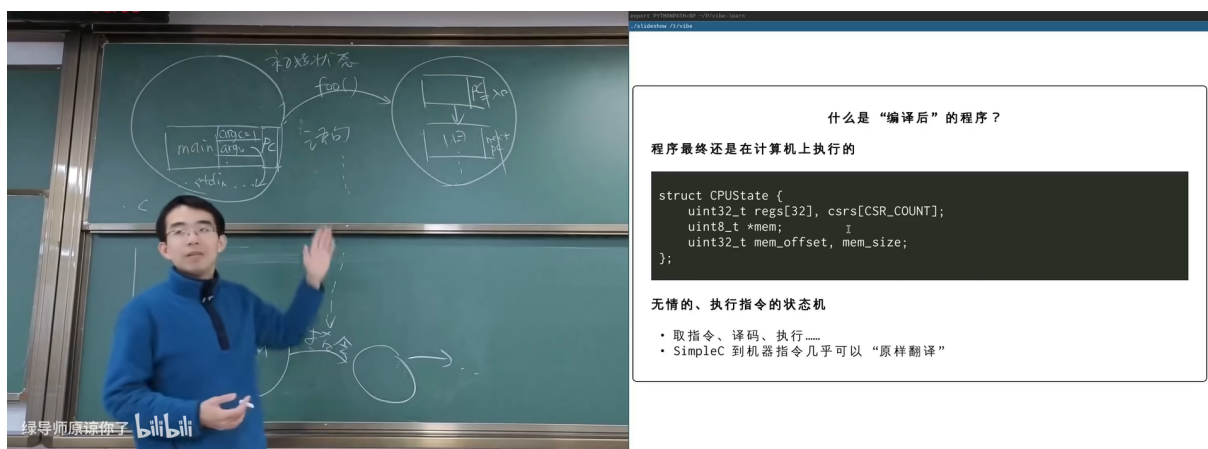
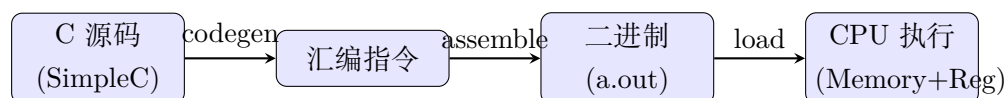


图 7: 编译器架构：从 C 源码到 CPU 可执行的状态机⁷

编译器做的事：把 .c 翻译成 a.out。广义的编译器包括汇编器和链接器。



jyy 的关键类比：“那个时候的编译器和今天的 transpiler 很像。最早期的编译器就是把语句原样翻译成指令，一条一条翻译。JavaScript 满地都是 transpiler——做一些简单的语法变化就可以把新版本变成旧版本。”

```
1 struct CPUState {
2     uint32_t regs[32];    // 寄存器组
3     uint8_t *mem;        // 内存
4     uint32_t mem_of;     // 内存偏移
5 };
```

Listing 4: CPU 状态的 C 表示

⁷ 视频画面时间区间：00:29:30–00:30:15。

无情的执行指令的状态机

CPU 就是一个“无情的、执行指令的状态机”——取指令、译码、执行、更新 PC。编译器的任务是确保这个机械过程能产出正确结果。CPU 不理解“你想做什么”，只理解“下一条指令是什么”。

2.2 编译器优化

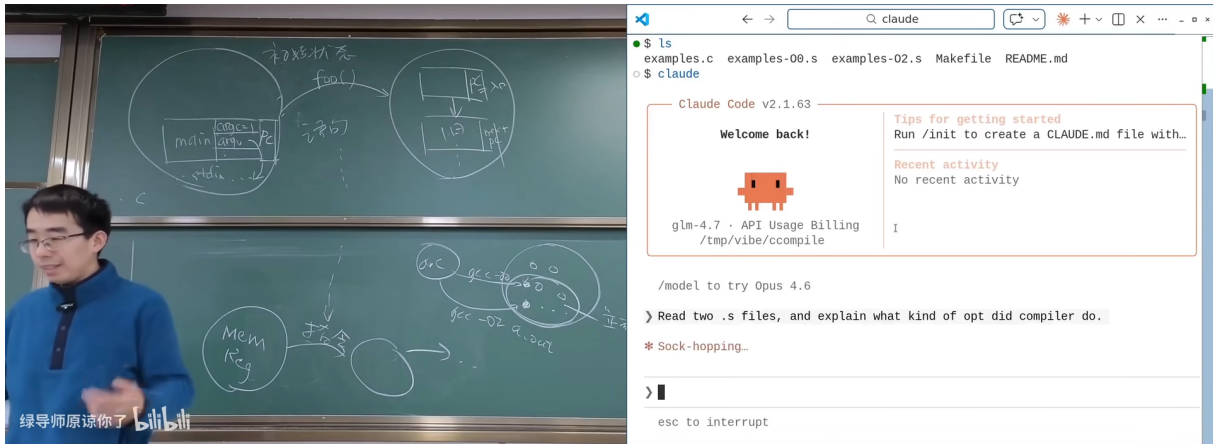


图 8: O0 vs O2: 8 条指令优化为 2 条⁸

jjy 用树莓派现场演示了 O0 和 O2 的差异：

冗余存储消除 (Redundant Store Elimination)：对同一变量的连续赋值，只保留最后一次。

常量传播：O0 时先存 24 再加 1；O2 时直接算出 25。“8 条指令变成 2 条。”

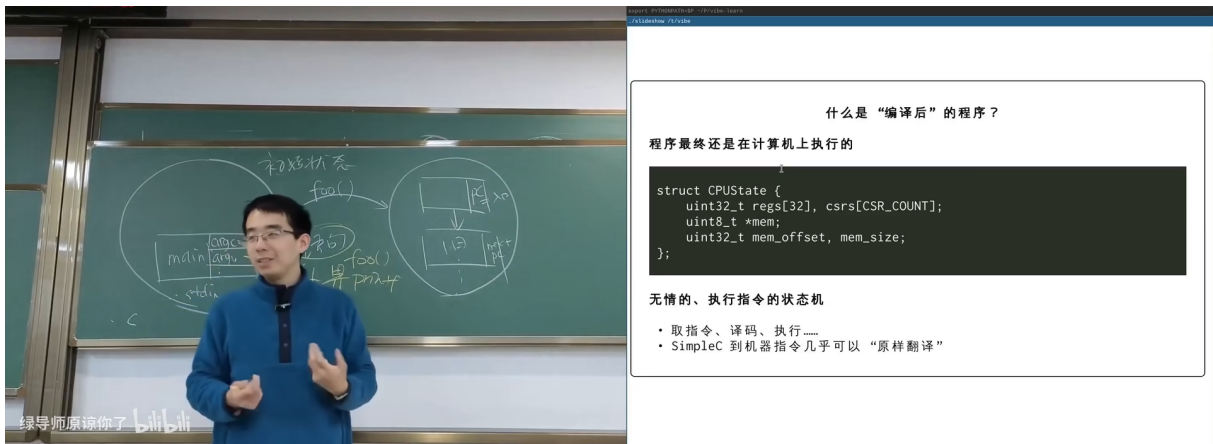


图 9: GOT 与 callee-saved 寄存器优化⁹

⁸ 视频画面时间区间：00:35:30–00:36:15。

⁹ 视频画面时间区间：00:38:00–00:38:45。

公共子表达式消除 + 常量传播 (CSE + TCP)

每次函数调用前都要重新加载全局变量 → 编译器用 callee-saved 寄存器保存，跨调用复用。GOT (Global Offset Table) 在位置无关代码中起关键作用。这些优化对操作系统性能至关重要——内核代码的热路径每条指令都在数。

2.3 本章小结

编译器 = 状态机翻译器：将“人类可读的状态机描述”翻译为“CPU 指令级状态机”，并在翻译过程中消除冗余 (RSE)、合并计算 (CSE)、传播常量。理解编译器做了什么，是理解程序在操作系统上真正如何运行的前提。

3 最小 Hello World

jyy: “你们都从 `printf` 开始的。但在 `main` 函数执行之前，机器其实已经执行了很多很多别的指令。那你不能写一个最小的程序？”

3.1 标准 `hello.c` 的真面目

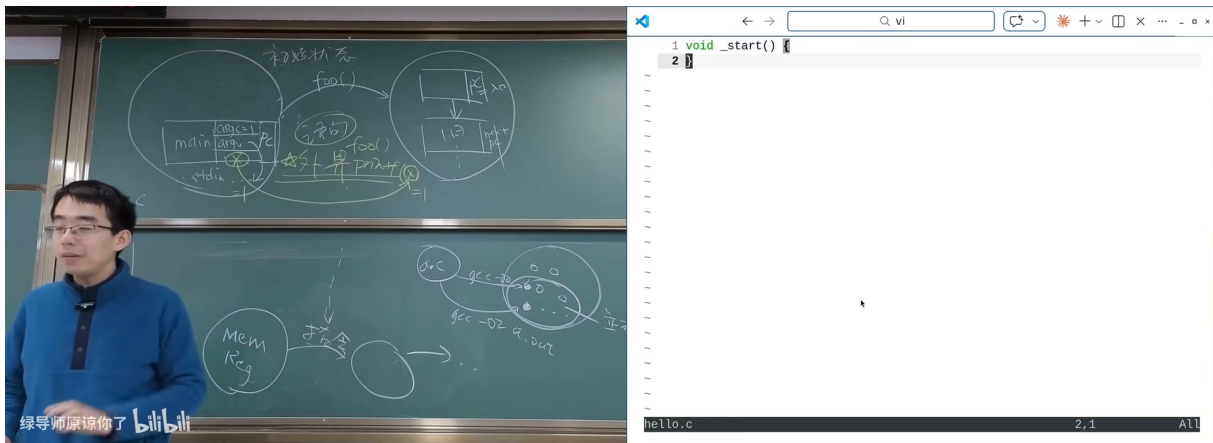


图 10: `gcc -c → objdump → ld`: 揭示标准 `hello` 的隐藏复杂性¹⁰

```
1 $ gcc hello.c           # 标准编译
2 $ ls -la a.out          # 文件有 70KB!
3 $ objdump -d a.out      # 里面有大量你不认识的代码
4 $ gcc -c hello.c        # 只编译不链接
5 $ ld hello.o            # 直接链接会失败:
6 # ld: warning: cannot find entry symbol _start
7 # ld: hello.o: in function `main':
8 # hello.c:(.text+0x10): undefined reference to `puts'
```

Listing 5: 编译、查看、链接的完整流程

¹⁰视频画面时间区间: 00:44:00–00:44:45。

为什么 ld hello.o 会失败？

两个原因：(1) 找不到入口符号 `_start`——`main` 不是程序的真正入口！(2) `puts` 是 C 标准库函数，需要链接 `libc`。

这揭示了一个重要事实：你写的 `main()` 被操作系统和运行时库“包裹”着执行。在 `main` 之前有 `_start` → `__libc_start_main` → `main` 的调用链。

3.2 minimal.S：不依赖任何库的最小程序

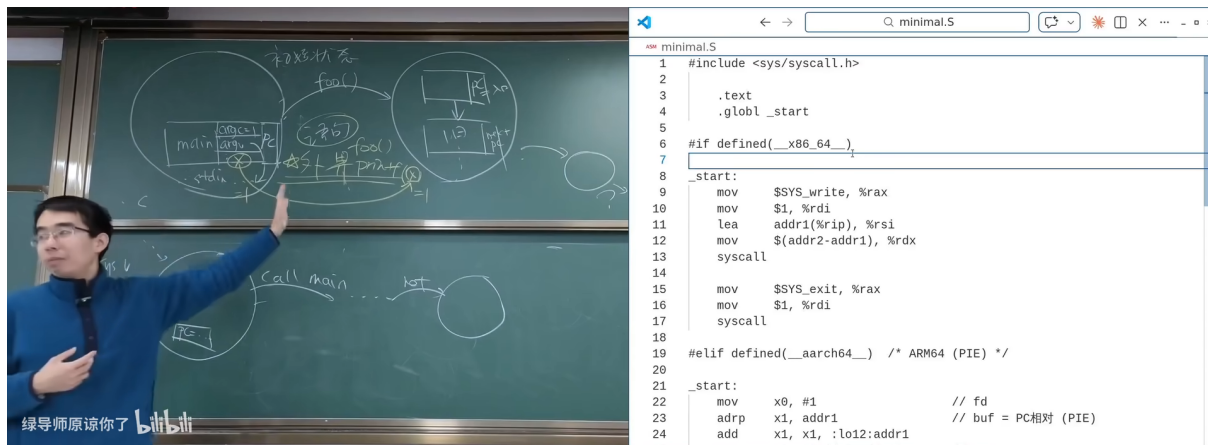


图 11: minimal.S：直接使用系统调用，不依赖任何库¹¹

```
1 #include <sys/syscall.h>
2 .text
3 .global _start
4
5 _start:
6     mov $SYS_write, %rax    // syscall number = 1
7     mov $1, %rdi           // fd = 1 (stdout)
8     lea addr(%rip), %rsi   // buffer address
9     mov $14, %rdx          // length
10    syscall                // invoke kernel!
11
12    mov $SYS_exit, %rax     // syscall number = 60
13    mov $0, %rdi           // exit code = 0
14    syscall
```

Listing 6: minimal.S — x86-64 版本

jyy 现场演示：“如果我执行这个 `minimal`，就会输出 `Hello OS World`。如果把退出码参数改成 10，程序的返回编号就变成 10。”

¹¹视频画面时间区间：00:48:00–00:49:00。

程序与操作系统的真正接口

minimal.S 证明了：一个完整程序可以**不依赖任何库**，只用系统调用指令就能运行。程序与操作系统之间的唯一接口不是 `printf`、不是 `main`，而是一条**硬件指令**：

`syscall (x86-64)    ecall (RISC-V)    svc (AArch64)`

3.3 系统调用：请求操作系统服务

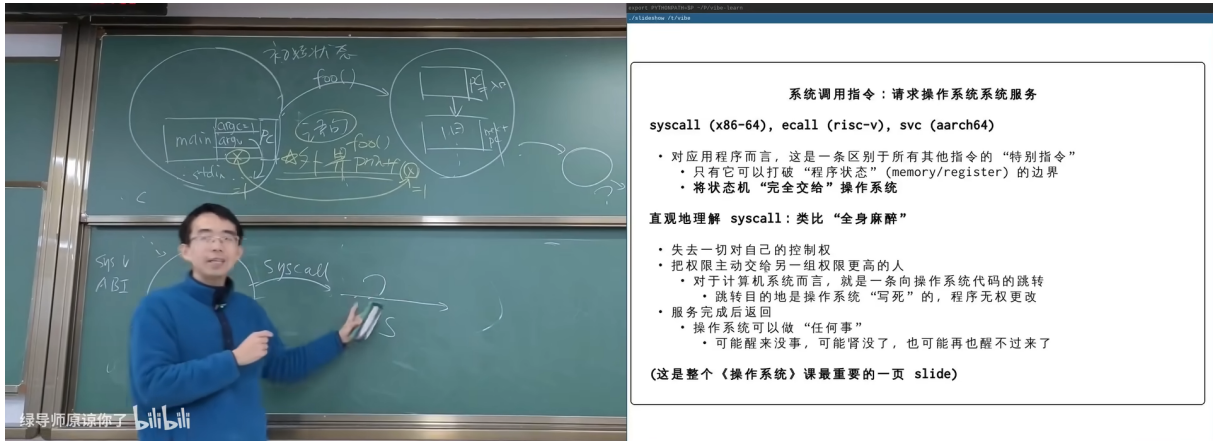


图 12: 系统调用指令——这是整个《操作系统》课最重要的一页 slide¹²

jyy 原话：“这是整个《操作系统》课最重要的一页 slide。”

syscall 的本质：“全身麻醉”

直接地理解 `syscall`：类比“全身麻醉”——

1. 失去一切对自己的控制权
2. 把权限交给操作系统（“医生”）
3. 操作系统检查请求、执行操作系统代码的回调
4. 一直到操作系统通知让程序“苏醒”，程序从暂停处恢复执行
5. 操作系统可以“毒杀”你——如果你做了非法操作

`write()` $\neq$ `syscall`

很多人以为 `write()` 就是系统调用。严格来说不是——`write()` 是 C 库函数，它**内部**才执行了 `syscall` 指令。真正的系统调用是那条汇编指令。这个区分在安全分析和 `strace` 使用中至关重要。

¹² 视频画面时间区间：00:52:30–00:53:30。

3.4 本章小结

程序不是从 `main` 开始的，而是从 `_start` 开始的。程序与操作系统的唯一接口是系统调用指令。`minimal.S` 只用两个 `syscall` (`write` + `exit`) 就实现了完整的 Hello World——这是操作系统课最核心的演示。

4 开箱真实应用程序

jyy：“实际上，操作系统上所有的程序和 `minimal` 没有本质区别。所有程序都共享完全相同的一套 `system call`。在操作系统面前，众生平等。”

4.1 操作系统面前众生平等

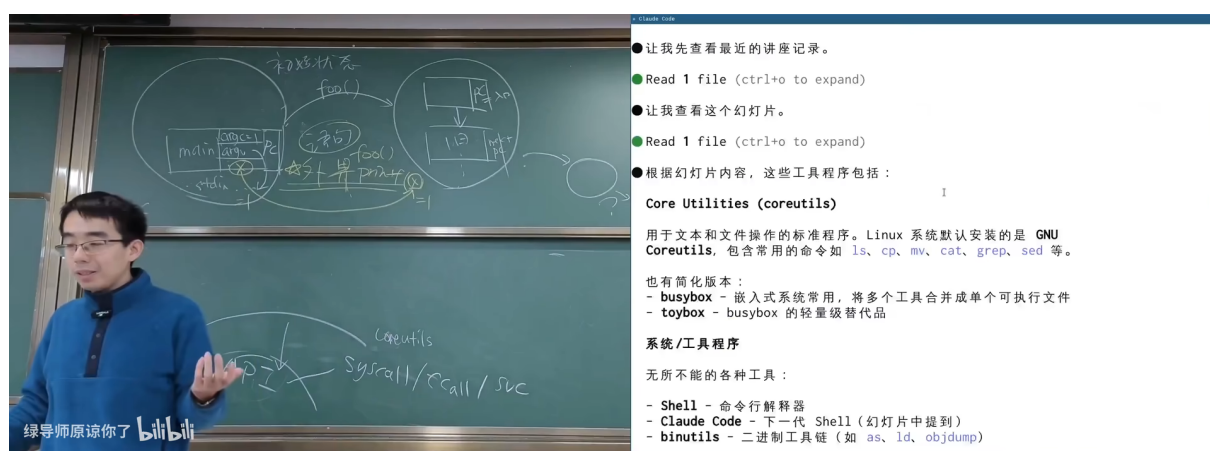


图 13: 所有程序在 OS 面前都是一样的——都通过 `syscall` 交互¹³

jyy 指着桌面上的各种程序——终端、VS Code、窗口管理器：“所有运行进程看起来都是一样的。当然可能有些调度策略上的区别——有个机制叫 `nice`，越好的人得到 CPU 越少。但从概念上，所有程序都是一样的，都和你的 `minimal` 完全一样。”

jyy 列举了日常使用的程序：VS Code、Cursor、Claude Code——“我现在觉得没有 Claude Code 我已经不会编程了。虽然我觉得现在的编程还不是进化到完全体。”

¹³ 视频画面时间区间：01:01:00–01:01:45。

4.2 strace: 系统调用追踪

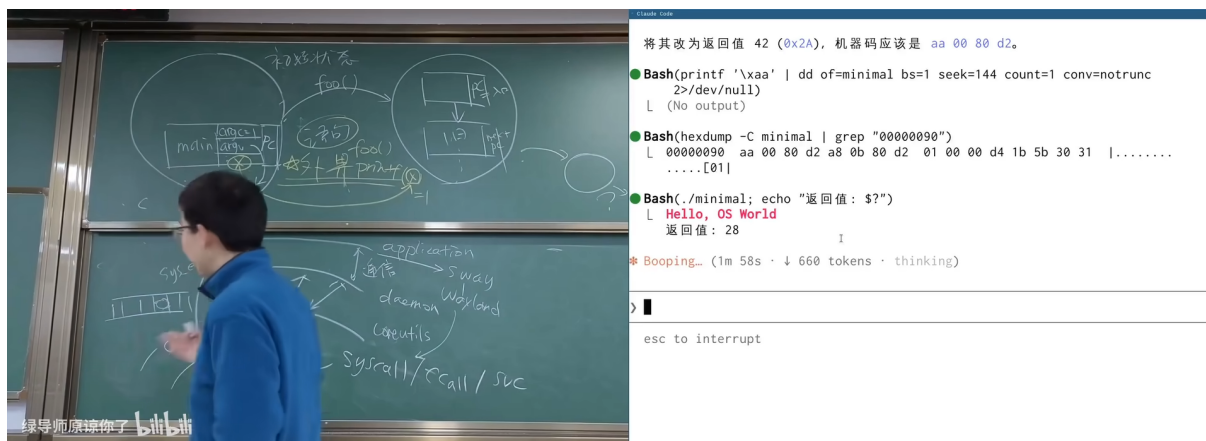


图 14: strace 追踪程序执行：观测状态机的“对外接口”¹⁴

strace

strace = System call trace。它拦截并记录程序发出的**每一个系统调用**。

理解程序是如何与操作系统交互的——观测状态机执行的“对外接口”。当你不理解一个程序在做什么时，strace 给你答案。

4.3 开箱 gcc：编译器到底做了什么？

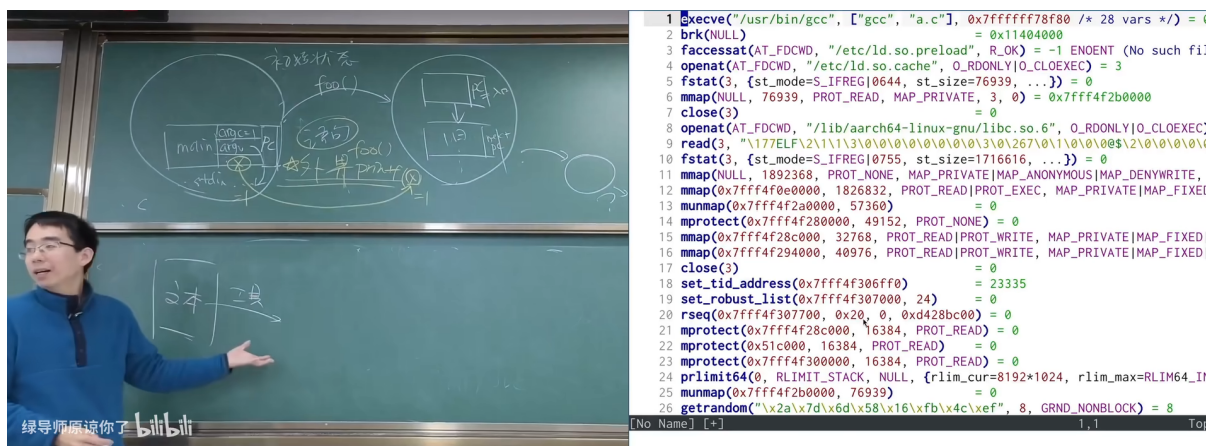


图 15: strace 追踪 gcc：从 execve 到 mmap 的完整系统调用记录¹⁵

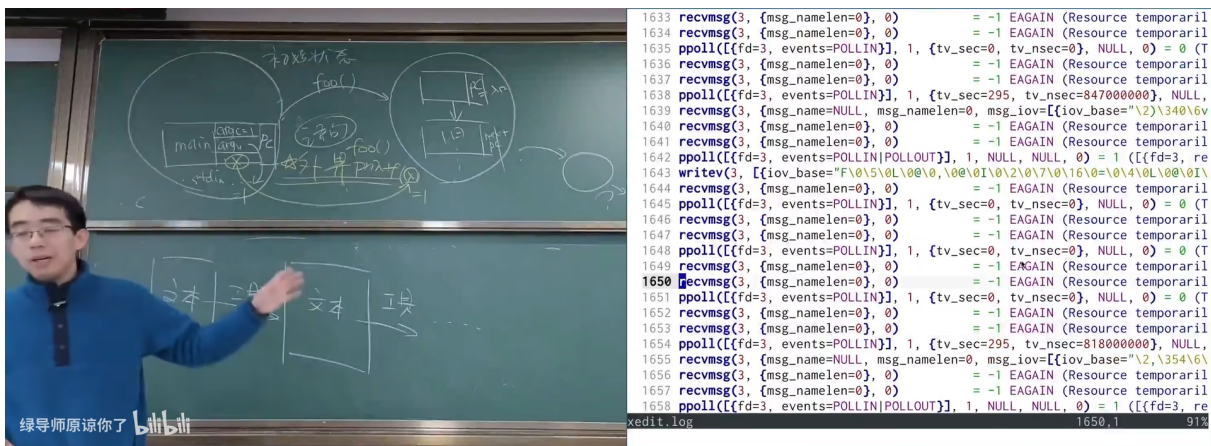
jyy 用 strace 追踪了 gcc "a.c" 的完整过程：

```
1 1 execve("/usr/bin/gcc", ["gcc", "a.c"], ...) = 0
2 2 [pid 23336] execve("../cc1", [...]) # C -> assembly
3 3 [pid 23338] execve("../as", [...]) # assembly -> .o
4 ...
5 10 [pid 23339] execve("../collect2", [...]) # 收集
```

¹⁴ 视频画面时间区间：01:08:00–01:09:00。

¹⁵ 视频画面时间区间：01:11:30–01:12:15。

Listing 7: strace 追踪 gcc 编译过程 (精简)

图 16: 黑板上的完整应用架构: application → daemon → sway → syscall/OS¹⁶

关键发现:

- gcc 本身只是一个调度器——它自己不编译任何代码
- 真正干活的是 cc1 (C → 汇编)、as (汇编 → .o)、collect2+ld (链接)
- 一次编译背后有超过 10 个 execve 系统调用
- jyy 还让 AI 来分析 strace 输出——“AI 的能力就真的是出乎意料，它找到了 Hello World 的字节在哪里”

UNIX 哲学的活体标本

gcc 的架构就是 UNIX 哲学的活体标本：“做一件事，做好它”。复杂任务不是由一个巨大的单体程序完成，而是由多个小工具通过 execve 和管道组合而成。这也是为什么理解 execve 系统调用如此重要——它是程序组合的基础设施。

4.4 本章小结

strace 是理解程序行为的终极工具。通过它我们发现：(1) gcc 是一个调度器，不是编译器；(2) 所有程序在 OS 面前众生平等；(3) 即使是最复杂的应用，底层也只是 syscall 的序列。这就是“应用视角的操作系统”——从程序的行为反推 OS 提供了什么。

5 总结与延伸

5.1 讲者的核心信息

jyy 在第二讲构建了一个完整的认知链条：

程序 $\xrightarrow{\text{形式化}}$ 状态机 $\xrightarrow{\text{编译}}$ 指令序列 $\xrightarrow{\text{syscall}}$ 操作系统

¹⁶ 视频画面时间区间：01:23:00–01:24:00。

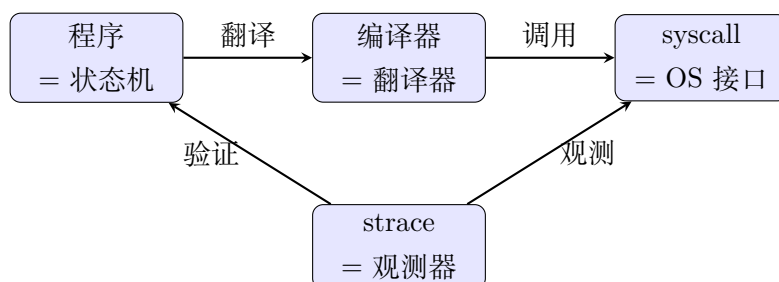
1. **程序 = 状态机**：变量 + 栈帧 + PC 构成状态，执行一条语句就是一次迁移
2. **编译器 = 翻译器**：将高级状态机描述翻译为 CPU 指令级状态机，并优化
3. **syscall = 唯一接口**：程序与 OS 之间只有这一条“安全沟”
4. **strace = 观测器**：让你看到程序和 OS 之间的每一次交互

5.2 结构化提炼

四个核心 Takeaway

1. **SimpleC 让状态机可触摸**：不是抽象概念，而是可逐步追踪的真实执行。递归转迭代不是技巧——是状态机理论的直接推论。
2. **minimal.S 揭示了真正的接口**：程序从 `_start` 开始而非 `main`；与 OS 的接口是 `syscall` 指令而非 C 库函数。所有中间层都是抽象。
3. **syscall 是“全身麻醉”**：程序交出控制权，OS 可以做任何事（包括杀死你）。这个类比是理解用户态/内核态的最佳直觉。
4. **strace 是最强调试工具**：当你不理解程序行为时，`strace` 给你完整的系统调用日志。它揭示了 gcc 不是编译器（而是调度器）这样反直觉的事实。

5.3 跨章节关联



5.4 与第 01 讲的关联

第 01 讲说“操作系统是管理硬件和软件的软件”。第 02 讲具体化了这句话：“管理软件” = 管理程序的状态机（进程）；“接口” = 系统调用指令；“观测” = `strace` 让你亲眼看到管理过程。

5.5 开放问题

1. 多个状态机（进程）同时运行时，OS 如何调度？→ 后续的“并发”章节
2. `execve` 在 OS 内部做了什么？→ 后续的“进程管理”章节
3. jyy 提到的 `nice` 机制——“越好的人得到 CPU 越少”——背后的调度算法是什么？
4. 如果 Claude Code 也是一个程序，它的 `strace` 输出会是什么样子？

5.6 拓展阅读

- OSTEP: <https://pages.cs.wisc.edu/~remzi/OSTEP/>
- PicoC: <https://github.com/jpoirier/picoc>
- jyy 的课程 Wiki: <https://jyywiki.cn/>